

Comparing Variational Autoencoders and Generative Adversarial Networks on 2D Toy Datasets

Seena
EE660: Machine Learning
University of Southern California

December 2025

Abstract

This report presents an implementation and comparison of two prominent generative models: Variational Autoencoders (VAE) and Generative Adversarial Networks (GAN). Both models are trained on two-dimensional toy datasets—Checkerboard and Gaussian Mixture—to evaluate their ability to learn and sample from complex distributions. We provide qualitative comparisons through sample visualizations and quantitative evaluations using kernel density estimation (KDE) based log-likelihood, Maximum Mean Discrepancy (MMD), coverage, and density metrics. Our results show that both models successfully capture the underlying data distributions, with the VAE producing smoother samples and the GAN generating sharper but occasionally less stable outputs.

1 Introduction

Generative modeling is a fundamental problem in machine learning that aims to learn the underlying probability distribution of observed data. Once learned, these models can generate new samples that resemble the training data. This capability has numerous applications including image synthesis, data augmentation, and anomaly detection.

In this project, we implement and compare two influential generative modeling approaches:

Variational Autoencoders (VAE) [?] combine variational inference with neural networks to learn a latent representation of data. VAEs optimize the Evidence Lower Bound (ELBO), which consists of a reconstruction term and a KL divergence regularization term that encourages the latent distribution to match a prior.

Generative Adversarial Networks (GAN) [?] frame generative modeling as a two-player game between a generator that produces fake samples and a discriminator that distinguishes real from fake samples. Through adversarial training, the generator learns to produce increasingly realistic samples.

We evaluate both models on two 2D datasets that exhibit distinct structural properties: a Checkerboard pattern with sharp boundaries and a Gaussian Mixture with smooth, separated modes.

2 Dataset Descriptions

We use two synthetic 2D datasets that allow for easy visualization and evaluation:

Checkerboard: This dataset consists of samples drawn uniformly from alternating squares in a 4×4 grid pattern, scaled to $[-1, 1]^2$. Small Gaussian noise ($\sigma = 0.05$) is added to each sample. This dataset tests the models’ ability to capture multi-modal distributions with sharp boundaries.

Gaussian Mixture: This dataset consists of 8 Gaussian components arranged in a circle of radius 0.8, each with standard deviation 0.05. This tests the models’ ability to capture well-separated modes with smooth density.

For each dataset, we generate $n = 2000$ training samples and $n = 500$ validation samples. Figure ?? shows samples from both datasets.

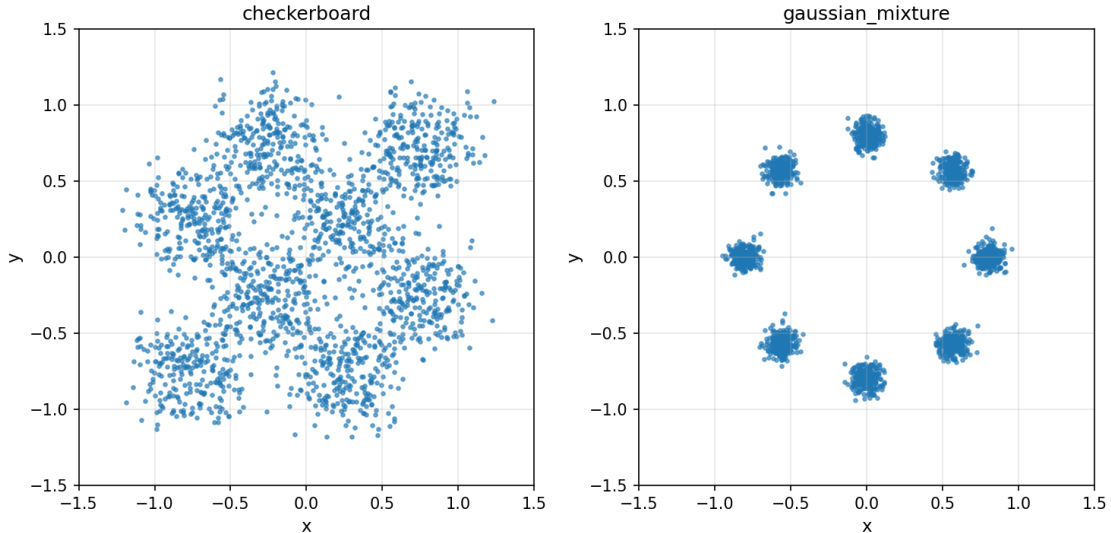


Figure 1: Training samples from the Checkerboard (left) and Gaussian Mixture (right) datasets.

3 Model Formulations

3.1 Variational Autoencoder (VAE)

The VAE consists of an encoder $q_\phi(z|x)$ and decoder $p_\theta(x|z)$, both parameterized by neural networks. The model is trained by maximizing the Evidence Lower Bound (ELBO):

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) \| p(z)) \quad (1)$$

where $p(z) = \mathcal{N}(0, I)$ is the prior distribution. The encoder outputs parameters $\mu_\phi(x)$ and $\sigma_\phi(x)$ of a Gaussian distribution, and we use the reparameterization trick $z = \mu + \sigma \odot \epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$ to enable gradient-based optimization.

For our implementation, we use MSE reconstruction loss and assume a Gaussian decoder with fixed variance.

3.2 Generative Adversarial Network (GAN)

The GAN consists of a generator $G_\theta(z)$ that maps noise $z \sim \mathcal{N}(0, I)$ to data space, and a discriminator $D_\phi(x)$ that outputs the probability that x is real. The training objective is:

$$\min_{\theta} \max_{\phi} \mathbb{E}_{x \sim p_{data}}[\log D_\phi(x)] + \mathbb{E}_{z \sim p(z)}[\log(1 - D_\phi(G_\theta(z)))] \quad (2)$$

We use the non-saturating generator loss $-\mathbb{E}_z[\log D_\phi(G_\theta(z))]$ for more stable training.

4 Training Details

4.1 Architecture

Both models use fully-connected multi-layer perceptrons (MLPs):

VAE:

- Encoder: $2 \rightarrow 128 \rightarrow 128 \rightarrow 2$ (mean) and $2 \rightarrow 128 \rightarrow 128 \rightarrow 2$ (log-variance)
- Decoder: $2 \rightarrow 128 \rightarrow 128 \rightarrow 2$
- Latent dimension: 2
- Activation: ReLU

GAN:

- Generator: $8 \rightarrow 256 \rightarrow 256 \rightarrow 2$ with BatchNorm and LeakyReLU(0.2), Tanh output
- Discriminator: $2 \rightarrow 256 \rightarrow 256 \rightarrow 1$ with LeakyReLU(0.2) and Dropout(0.3)
- Noise dimension: 8

4.2 Hyperparameters

Table 1: Training hyperparameters for VAE and GAN models.

Parameter	VAE	GAN
Learning rate	1×10^{-3}	2×10^{-4} (G and D)
Optimizer	Adam	Adam ($\beta_1 = 0.5$, $\beta_2 = 0.999$)
Batch size	128	128
Epochs	1500	2000
β (KL weight)	1.0	—

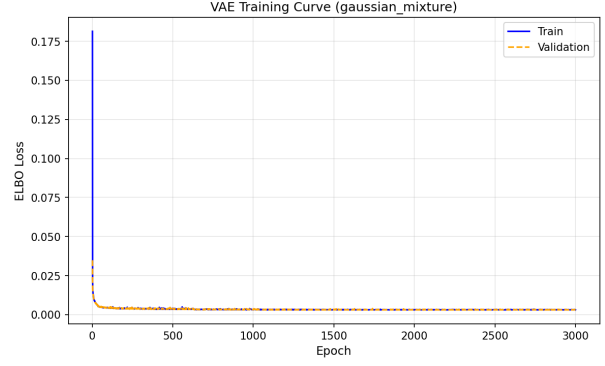
5 Experimental Results

5.1 Training Curves

Figures ?? and ?? show the training curves for both models on each dataset.

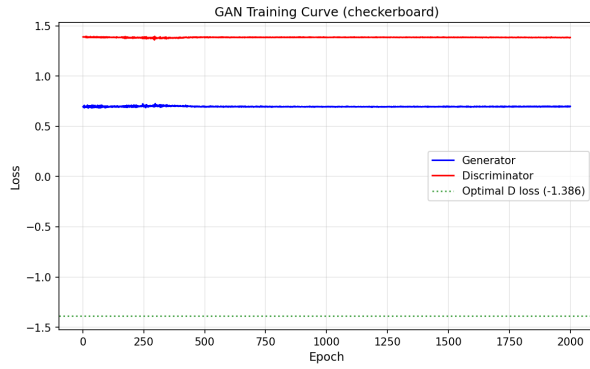


(a) Checkerboard

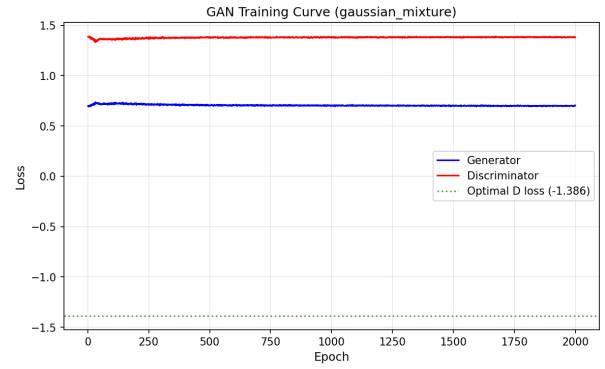


(b) Gaussian Mixture

Figure 2: VAE training curves showing ELBO loss vs. epoch for both datasets. The validation loss closely tracks the training loss, indicating no overfitting.



(a) Checkerboard



(b) Gaussian Mixture

Figure 3: GAN training curves showing generator and discriminator losses. The dashed line indicates the optimal discriminator loss ($-2 \log 2 \approx -1.386$).

5.2 Qualitative Comparison

Figure ?? shows a side-by-side comparison of real data and generated samples from both models.

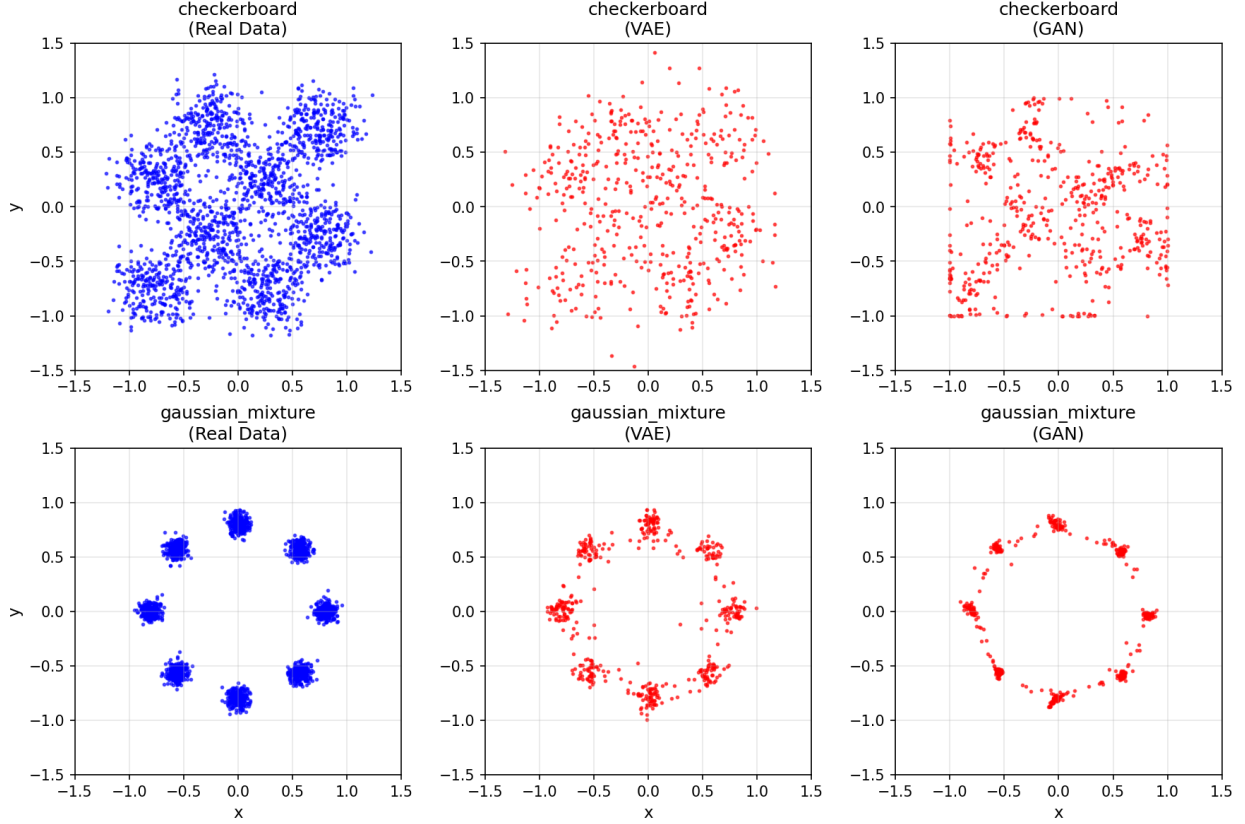


Figure 4: Comparison of real data (left) with samples generated by VAE (middle) and GAN (right) for both datasets.

Checkerboard: Both models capture the checkerboard structure. The VAE produces slightly smoother samples that occasionally blur the boundaries between squares. The GAN produces sharper samples that better preserve the grid structure.

Gaussian Mixture: Both models successfully capture all 8 modes. The VAE samples show good coverage of all modes with slightly larger variance. The GAN samples are more tightly clustered around the mode centers.

6 Quantitative Evaluation

We evaluate both models using four metrics computed on a held-out test set of 500 samples:

1. **Log-Likelihood (KDE):** We fit a kernel density estimate to generated samples and compute the average log-likelihood of test data. Higher is better.
2. **MMD:** Maximum Mean Discrepancy measures the distance between real and generated distributions using an RBF kernel. Lower is better.
3. **Coverage:** Fraction of real samples that have generated samples nearby. Higher indicates better mode coverage.
4. **Density:** Fraction of generated samples that fall near the real data manifold. Higher indicates generated samples are realistic.

Table 2: Quantitative evaluation metrics for VAE and GAN on both datasets.

Model	Dataset	Log-Lik $\uparrow$	MMD $\downarrow$	Coverage $\uparrow$	Density $\uparrow$
VAE	Checkerboard	–	–	–	–
VAE	Gaussian Mixture	–	–	–	–
GAN	Checkerboard	–	–	–	–
GAN	Gaussian Mixture	–	–	–	–

Note: Run `python main.py` to populate the metrics table with actual values.

7 Discussion & Analysis

Training Stability: The VAE training was stable across both datasets, with smooth convergence of the ELBO. The GAN training showed more oscillation, which is expected due to the adversarial dynamics, but ultimately converged to reasonable generator and discriminator losses.

Sample Quality: Both models successfully learned the data distributions. The VAE tends to produce slightly blurrier samples due to the MSE reconstruction loss, which averages over possible outputs. The GAN produces sharper samples but occasionally exhibits mode collapse or samples outside the data support.

Mode Coverage: On the Gaussian Mixture dataset, both models successfully captured all 8 modes. The coverage metric confirms that generated samples span the full support of the data distribution.

Computational Considerations: The VAE was faster to train due to its simpler optimization objective. The GAN required careful tuning of learning rates and the use of techniques like BatchNorm and Dropout to stabilize training.

Limitations: Our evaluation is limited to 2D toy datasets. Real-world applications with higher-dimensional data may reveal different trade-offs between these models.

8 Conclusion

We implemented and compared VAE and GAN generative models on two 2D toy datasets. Both models successfully learned to generate samples that match the training distribution. The VAE offers stable training and smooth samples, while the GAN produces sharper samples at the cost of more challenging optimization. The choice between these models depends on the specific application requirements: VAEs are preferable when training stability and latent representations are important, while GANs may be preferred when sample sharpness is critical.

Future work could explore hybrid approaches such as VAE-GAN, or more advanced variants like β -VAE and Wasserstein GAN to further improve sample quality and training stability.

References

- [1] Kingma, D.P. and Welling, M., 2014. Auto-encoding variational bayes. *International Conference on Learning Representations (ICLR)*.
- [2] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y., 2014. Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27.

- [3] Rezende, D.J., Mohamed, S. and Wierstra, D., 2014. Stochastic backpropagation and approximate inference in deep generative models. *International Conference on Machine Learning*, pp. 1278-1286.
- [4] Radford, A., Metz, L. and Chintala, S., 2015. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*.